

CampusEats Orders REST Service

CS543 · Web Services · Assignment 5: HTTP Methods & Headers

Team ID: 5 | Team Leader: Yash Namdev (Roll / Reg. No.: 20252651066)

Team Members: Yash Namdev (20252651066), Arpit Tamrakar (20252651012), Harsh Jain (20252651022), Anupam Rai (20252651011), Saurabh Singh Chauhan (20252651049)

This submission completes **Assignment 5 (HTTP Methods & Headers)** for the CampusEats Orders service. All HTTP verb semantics, query string parameters, custom header protocols, caching, rate limiting, and safe retries are implemented in Python (WSGI) with zero external runtime dependencies.

Part A — Methods & The Message

A1–A5: CampusEats Method Map

Action	Method + URL	Safe	Idempotent	Notes
List/filter/sort/page orders	GET /orders?status=&sort=&page=&pageSize=	Yes	Yes	Pure read with query string params. Stays a pure read; never alters server state.
Read one order	GET /orders/{orderId}	Yes	Yes	Returns cached representation with ETag and Cache-Control. Supports conditional GET.
Create an order	POST /orders	No	No*	Creates order; returns 201 Created + Location. *Made retry-safe via Idempotency-Key.
Change delivery address	PATCH /orders/{orderId}	No	Yes	Modifies deliveryAddress. Protected by mandatory If-Match ETag against lost updates.
Delete an order	DELETE /orders/{orderId}	No	Yes	Removes resource; returns 204 No Content. Protected by If-Match.
Request order cancellation	POST /orders/{orderId}/cancellation	No	No	Non-CRUD action modeled as POST to sub-resource (never /cancelOrder). Returns 202 Accepted.
Discover supported methods	OPTIONS /orders/{orderId}	Yes	Yes	Returns Allow header listing permitted verbs; completes CORS preflight.

Constrained Client Fallback (A5): If a restricted client cannot send PATCH or DELETE, it may send POST with x-HTTP-Method-Override: PATCH OR DELETE as a documented fallback.

Part B — Headers on Every Message

D2: Request & Response Headers Table

Endpoint	Required / Accepted Request Headers	Important Response Headers Set & Purpose
GET /orders	Authorization: Bearer <token>, optional Accept, Accept-Encoding	Content-Type: application/json, Cache-Control: private, max-age=30, Content-Encoding: gzip, rate-limit headers, CORS & security headers
POST /orders	Authorization, Content-Type: application/json, Idempotency-Key, Accept	201 Created, Location: /orders/{id}, ETag: "<hash>", Cache-Control: no-store, rate-limit headers, CORS & security headers
GET /orders/{id}	Authorization, optional If-None-Match, Accept	200 OK Or 304 Not Modified (no body), ETag, Cache-Control: private, max-age=60, CORS & security headers
PATCH /orders/{id}	Authorization, Content-Type, required If-Match, optional override	200 OK (Or 412 Precondition Failed), updated ETag, Cache-Control: no-store
DELETE /orders/{id}	Authorization, required If-Match, optional override	204 No Content (Or 412 Precondition Failed), Cache-Control: no-store
POST /orders/{id}/cancellation	Authorization, Content-Type: application/json	202 Accepted (Or 409 Conflict, 422 Unprocessable), Cache-Control: no-store
OPTIONS	Browser preflight headers (Origin, Access-Control-Request-Method)	204 No Content, Allow, full Access-Control-* CORS suite

Part C — Caching & Safe Retries

C4: The Safe-Retry Plan

Risky Endpoint	Inherent Risk	Mechanism Applied	Justification
POST /orders	Network drop during payment or response; client retries. Duplicate	Idempotency-Key	The server atomically links the key to the original order. Retries return the

	risks charging student twice or cooking duplicate meals.		cached 201 response and Location without repeating charges or kitchen dispatch.
GET /orders/{id}	Bandwidth waste and redundant parsing on mobile network when entity is unchanged.	If-None-Match	Client transmits stored ETag. Matching ETag returns 304 Not Modified with zero-byte body, saving cellular data and battery.
PATCH /orders/{id}	Two concurrent dispatchers/customers updating delivery address simultaneously (Lost Update problem).	If-Match	Requires client to submit current ETag. If another editor updated the order first, server rejects with 412 Precondition Failed instead of overwriting.
POST /orders/{id}/cancellation	Duplicate cancellation requests or attempting to cancel already-completed orders.	State 409 / 422	Repeated cancellation returns 409 Conflict. Disallowed lifecycle state returns 422 Unprocessable Content.

A6: Fully Labelled Complete HTTP Exchange

Request — HTTP/1.1

```

POST /orders HTTP/1.1
Host: 127.0.0.1:8001
User-Agent: curl/8.5.0
Accept: application/json
Content-Type: application/json
Authorization: Bearer demo-token
Idempotency-Key: a5-demo-key-001
Content-Length: 119

and Body
{"customerId":"stu-102","items":[{"menuItemId":"menu-42","quantity":2}],"deliveryAddress":"Hostel A, Room 210"} <-- JSON Body

```

<-- Request Line (Method, URI, Protocol Version)
 <-- Host Header (Mandatory in HTTP/1.1)
 <-- Client User-Agent Header
 <-- Content Negotiation: Requesting JSON
 <-- Media Type of Payload
 <-- Bearer Authentication Credential
 <-- Idempotency Token for Safe Retries
 <-- Body Length in Bytes
 <-- Empty Line (CRLF) Separating Headers

Response — HTTP/1.0 (WSGI Development Server)

```

HTTP/1.0 201 Created
Reason)
Date: Tue, 22 Sep 2026 03:06:53 GMT
Server: WSGIServer/0.2 CPython/3.12.3
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PATCH, ...
Access-Control-Allow-Headers: Authorization, ...
Access-Control-Expose-Headers: Location, ETag, ...
Vary: Accept, Accept-Encoding, Origin
X-Content-Type-Options: nosniff

```

<-- Status Line (Protocol, Status Code, Reason)
 <-- Date Header
 <-- Server Header
 <-- Response Media Type
 <-- CORS: Permitted Caller Origins
 <-- CORS: Allowed HTTP Verbs
 <-- CORS: Permitted Request Headers
 <-- CORS: Headers Exposable to Client JS
 <-- Cache Invalidation Directives
 <-- Security: MIME Sniffing Prevention

Strict-Transport-Security: max-age=31536000	<-- Security: Enforce HTTPS
X-RateLimit-Limit: 50	<-- Rate Limit: Max Requests in Window
X-RateLimit-Remaining: 49	<-- Rate Limit: Remaining Requests
Location: /orders/ord-f8a326caa1a1	<-- Canonical URI of New Resource
Cache-Control: no-store	<-- Cache Directive: Do Not Cache Mutation
ETag: "0aa5da47aa253af6"	<-- Entity Tag Fingerprint of
Representation	
Content-Length: 209	<-- Body Length in Bytes
	<-- Empty Line (CRLF) Separating Headers
and Body	
{ "id": "ord-f8a326caa1a1", "customerId": "stu-102", "items": [{ "menuItemId": "menu-42", "quantity": 2 }], "deliveryAddress": "Hostel A, Room 210", "status": "PAYMENT_PENDING", "createdAt": "2026-09-22T03:06:53.004819+00:00" }	

HTTP Version Note: The client specifies HTTP/1.1. The standard Python `wsgiref.simple_server` responds with HTTP/1.0 close-on-finish semantics. In production behind Nginx/TLS, this is upgraded to HTTP/1.1 or HTTP/2.

Required Answers to Assignment Questions

1. For three of your endpoints, give the method, the success status, and the single response header that matters most — and why.

- **POST /orders: 201 Created;** Location: /orders/{orderId} matters most because the client cannot predict the generated order ID; Location provides the canonical resource URI to inspect or track.
- **GET /orders/{orderId}: 200 OK;** ETag matters most because it cryptographically fingerprints the representation, powering 304 conditional caching and 412 concurrency protection.
- **DELETE /orders/{orderId}: 204 No Content;** Cache-Control: no-store matters most because intermediate caches must immediately invalidate any stored copies and never serve stale cached deleted items.

2. Which of your endpoints are safe, and which are idempotent? Which one is neither, and how did you make it retry-safe?

Safe endpoints: GET /orders, GET /orders/{id}, OPTIONS. Idempotent endpoints: GET, OPTIONS, PATCH /orders/{id}, DELETE /orders/{id}. Neither safe nor idempotent: POST /orders and POST /orders/{id}/cancellation. POST /orders was made retry-safe via a mandatory Idempotency-Key header; repeated submissions return the cached original 201 response and Location without duplicate payment processing or food preparation.

3. Show one ETag from your service, the request that returns 304, and the write that returns 412. What does each save or prevent?

Example ETag: "0aa5da47aa253af6".

- GET /orders/ord-f8a326caa1a1 with If-None-Match: "0aa5da47aa253af6" returns **304 Not Modified**. Saves: Eliminates redundant payload transmission, network latency, and client JSON parsing.
- PATCH /orders/ord-f8a326caa1a1 with If-Match: "stale-etag-99999" returns **412 Precondition Failed**. Prevents: Prevents the Lost Update problem where concurrent edits overwrite newer modifications.

4. You return 422 for one case and 400 for another. Give the exact request that triggers each, and explain the difference.

- **400 Bad Request:** POST /orders with { "customerId": "stu-102" } (missing required fields items and

deliveryAddress).

- **422 Unprocessable Content:** POST /orders/{id}/cancellation on an already cancelled or delivered order.

Difference: 400 signifies a **syntactic/structural defect** (malformed payload or missing required schema fields).

422 signifies a **semantic/domain refusal** (the JSON structure is valid, but business logic rejects the state transition in the entity's current state).

5. A browser page on another origin calls your API and is blocked — yet your server logs show a 200. Who blocked it, and which response header fixes it?

The **client browser** blocked it under the Same-Origin Policy (SOP). The server received the request, processed it, and returned 200, but because the response omitted cross-origin permissions, the browser prevented client-side JavaScript from reading the data. The Access-Control-Allow-Origin: * header (alongside OPTIONS preflight headers) fixes it.

6. Name one response where you set Cache-Control to allow caching and one where you must use no-store. Why each?

- Allow caching: GET /orders/{id} sets Cache-Control: private, max-age=60 because individual order details rarely change second-to-second; private caching allows the student's browser to reuse it while preventing public proxy caching.
- Must use no-store: POST /orders and PATCH /orders/{id} set Cache-Control: no-store because mutations and financial transactions must never be replayed from cache.

7. Your search endpoint is a GET. When would POST be the right choice instead, and what do you give up by switching?

POST is appropriate when query criteria become excessively large (e.g. hundreds of item IDs or complex nested filter trees exceeding URL length limits) or contain sensitive data that should not leak into browser history or URL logs. Switching to POST gives up automatic HTTP cacheability, browser back/forward navigation, URL bookmarkability, and link shareability.

8. Location appears on a 201 and on a 3xx. What does it point to in each case?

On **201 Created**, Location points to the persistent URI of the newly created resource. On **3xx Redirection**, Location points to the target URI where the client should re-dispatch its request next.

Verification & Test Suite Evidence

```
$ python3 run_tests.py
Running 15 test cases for CampusEats Orders API (Assignment 5)...
[PASS] test_cancellation_sub_resource
[PASS] test_conditional_delete_and_412
[PASS] test_conditional_get_returns_304
[PASS] test_conditional_patch_update_and_412
[PASS] test_content_negotiation_unsupported_accept_returns_406
[PASS] test_create_returns_201_location_etag_and_security_headers
[PASS] test_gzip_compression_on_large_payload
[PASS] test_idempotent_repeat_returns_original_order
[PASS] test_invalid_query_params_return_400
[PASS] test_malformed_body_uses_problem_shape
[PASS] test_method_override_patch_and_delete
```

```
[PASS] test_missing_token_returns_401_and_unknown_returns_404
[PASS] test_options_discovery_and_cors
[PASS] test_query_params_filtering_sorting_and_pagination
[PASS] test_rate_limiting_signal
```

```
=====
```

```
Results: 15 passed, 0 failed in 0.009s
```

```
=====
```

```
$ python3 validate_openapi.py openapi.yaml
```

```
OpenAPI document parsed successfully.
```

```
Structural checks completed: 0 errors.
```